

Exercises:

In these slides: (some with corresponding exercises in the textbooks)

1. Give a greedy algorithm for the **Picking Coins** problem that runs in $O(n \log t)$ time.
2. For the **Task Scheduling** problem
 - (a) Are the following greedy rules optimal? (if so prove it, otherwise give an example input to break it)
 - i. Earliest finish time first.
 - ii. Latest finish time first.
 - (b) For the greedy rule (earliest start time first) implement it in pseudocode and analyze the runtime. [GT 10.4, R-10.2]
3. For the **Minimum Spanning Tree** problem, implement the greedy rule (cheapest edge to grow the tree) in pseudocode and analyze the runtime. [CLRS 23.2-2]

Recommended from the textbooks:

[GT 10.4, A-10.1] ART GALLERY GUARDING. Given a line L representing a long, and real numbers $X = \{x_0, x_1, \dots, x_{n-1}\}$ specifying positions of paintings in this hallway. Suppose that a single guard can protect all the paintings within distance at most 1 of his or her position (on both sides). Design an algorithm for finding a placement of guards that uses the minimum number of guards to guard all the paintings with positions in X . (see [CLRS 16.2-5] for same problem phrased differently)

[CLRS 16.1-3] Not just any greedy approach to the activity-selection problem produces a maximum-size set of mutually compatible activities. Give an example to show that the approach of selecting the activity of least duration from among those that are compatible with previously selected activities does not work. Do the same for the approaches of always selecting the compatible activity that overlaps the fewest other remaining activities and always selecting the compatible remaining activity with the earliest start time.

[CLRS 23-4] Alternative minimum-spanning-tree algorithms In this problem, we give pseudocode for three different algorithms. Each one takes a connected graph and a weight function as input and returns a set of edges T . For each algorithm, either prove that T is a minimum spanning tree or prove that T is not a minimum spanning tree. Also describe the most efficient implementation of each algorithm, whether or not it computes a minimum spanning tree.

a. MAYBE-MST-A(G, w)

```

1  sort the edges into nonincreasing order of edge weights  $w$ 
2   $T = E$ 
3  for each edge  $e$ , taken in nonincreasing order by weight
4      if  $T - \{e\}$  is a connected graph
5           $T = T - \{e\}$ 
6  return  $T$ 
```

b. MAYBE-MST-B(G, w)

```

1   $T = \emptyset$ 
2  for each edge  $e$ , taken in arbitrary order
3      if  $T \cup \{e\}$  has no cycles
4           $T = T \cup \{e\}$ 
5  return  $T$ 
```

c. MAYBE-MST-C(G, w)

```

1   $T = \emptyset$ 
2  for each edge  $e$ , taken in arbitrary order
3       $T = T \cup \{e\}$ 
4      if  $T$  has a cycle  $c$ 
5          let  $e'$  be a maximum-weight edge on  $c$ 
6           $T = T - \{e'\}$ 
7  return  $T$ 
```